机器学习知识点

1. 机器学习基础

- **任务分类**：
  - **分类 (Classification)**：目标是预测离散标签（如：猫/狗、晴天/雨天）。
  - **回归 (Regression)**：目标是预测连续数值（如：具体的气温、房价）。
- **数据集划分**：
  - **训练集 (Training Set)**：用于模型真正“学习”和调整模型参数（如权重 W、偏置 b）。
  - **验证集 (Validation Set)**：用于模型评估和调整模型超参数（如学习率 η、K 值）。
  - **测试集 (Test Set)**：评估模型的泛化能力。
- **模型超参数**：学习率、正则化系数等属于手动设置或交叉验证调优的“超参数”。超参数调优的常用方法包括网格搜索、随机搜索及贝叶斯优化。
- **模型参数**：网络权重矩阵属于模型学习到的“参数”。每个神经元都有不同的权重和偏差值，权重和偏差都是神经网络的模型参数。
- **图像分类的挑战**：面临语义鸿沟、背景杂波（干扰）、视角变化、照明、变形、遮挡、类内变化等诸多问题。
- **算法三大支柱**：几乎所有的机器学习算法都由以下三部分组成：
  1. **表示 (Representation)**：设定候选函数空间（如线性模型、神经网络）。
  2. **评价 (Evaluation)**：设定衡量标准（如损失函数、准确率）。
  3. **最优化 (Optimization)**：寻找最优模型的搜索方法（如梯度下降）。

2. K 近邻算法(KNN)

- **核心机制**：KNN 是典型的“懒惰学习”，无显式训练阶段，仅存储数据；预测时直接计算样本间的距离。
- **距离度量**：
  - L1 (曼哈顿)距离：$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$。
  - L2 (欧氏)距离：$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$。
- **决策规则**：在分类任务中，KNN 从 K 个最近点中进行**多数投票**来决定最终标签。
- **参数敏感性与问题**：KNN 超参数（K值、距离度量）是人工设定的，而不是自动学习的，非常依赖于具体问题。模型预测结果高度依赖 K 值的选择（K 值通常选奇数以避免平局）。
- **交叉验证**：当数据量较少时，常使用 K 折交叉验证来最大化数据利用率。KNN 中的 N 折交叉验证：将数据分成 N 份，尝试每一份依次作为验证集，并计算结果的平均值。

3. 损失函数

- **交叉熵损失**：衡量预测概率与真实标签的差异，本质是惩罚模型对正确标签的“不自信”，强调正确分类的概率越大越好。
  - 公式为 $L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$。初始化时所有 s 近似相等，这时 loss 值是 $\log(C)$。
  - **极值范围**：最小值为 0（概率趋近 1），最大值为 $+\infty$。
- **SVM 损失**：核心在于“安全间距”，强调正确类别的得分必须比错误类别高出一定的安全间距，超过即损失为 0。
  - 公式为：$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$。
  - 数据集上的总损失为：$L = \frac{1}{N} \sum_{i=1}^{N} L_i$。初始化时所有 $s \approx 0$，这时 loss 值是 $N - 1$。

- **极值范围**：最小值为 0，最大值为 $+\infty$。
- **含正则化项的总损失**：引入正则化后的损失公式表达为：$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i + \lambda R(W)$。

4. 正则化

- **正则化的目的**：防止过拟合，抑制对训练数据中噪音的拟合；在权重上表达偏好；提高泛化能力（使模型变简单，以便在测试数据上更稳定）。
- **L1 正则化**：在原始损失上增加权重绝对值之和的惩罚项，倾向于产生稀疏权重矩阵，自带特征选择功能。具体公式为 $R_1(W) = \sum_k \sum_l |W_{k,l}|$。
- **L2 正则化 (权重衰减)**：在原始损失上增加权重平方和的惩罚项，限制权重数值过大以防止过拟合，倾向于让所有权重变小但非零，使模型更平滑。具体公式为 $R(W) = \sum_k \sum_l W_{k,l}^2$。注意：偏置项通常不参与正则化约束。梯度变化为：新梯度 = 原梯度 + λw_i。
- **弹性网络**：结合 L1 与 L2 正则化，公式为 $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$。

5. 神经网络与反向传播

- **线性分类器的局限性**
  - **决策边界**：只能绘制线性决策边界，无法妥善处理复杂的非线性数据分布（如环形、多簇分布）。
  - **模板限制**：每个类别只能学习单一模板，表达能力有限。
- **人工神经元结构**
  - **线性计算**：接收输入特征，加权求和并加上偏差，得到中间值 $z = Wx + b$。
  - **非线性激活**：将中间值 z 传入**激活函数**，引入非线性表达能力，得到最终输出 a。
- **神经网络架构**
  - **基本组成**：输入层 (Input Layer) $\rightarrow$ 隐藏层 (Hidden Layers) $\rightarrow$ 输出层 (Output Layer)。
  - **网络深度**：神经网络的深度 = 隐藏层 + 输出层。计算网络层数时**不包含输入层**。例如，“1个输入层 + 2个隐藏层 + 1个输出层”被称为**3层神经网络**。
- **学习目标**：给定网络结构后，通过算法找到一组最优参数（权重 W 和偏差 b），使得在所有训练数据上的**总体损失函数 L 达到最小**。
- 梯度：梯度指向损失函数上升最快的方向，为降低损失，参数需向梯度的反方向更新。
  - $\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$
  - $\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \frac{\partial A}{\partial Z}$
  - $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} X^T$
  - $\frac{\partial L}{\partial X} = W^T \frac{\partial L}{\partial Z}$
- **梯度检查**：在实践中，总是使用解析梯度，但是用数值梯度检查实现，这被称为梯度检查。
- **反向传播更新参数**：沿损失函数对参数梯度的负方向更新。公式为 $w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w}$ 或矩阵形式 $W^{(new)} = W^{(old)} - \eta \frac{\partial L}{\partial W}$，其中 η 为学习率。
- **小批量随机梯度下降**
  - **全批量痛点**：当训练集样本量 N 极大时，每走一步都要计算全部样本的总损失和梯度，极其耗费计算资源且缓慢。
  - **解决方案**：每次只从训练集中随机抽取一小批样本（通常大小为 32、64、128 或 256 等，称为批大小 Batch Size），用这批样本的梯度来近似代表整体的梯度。
  - **优势**：大幅提升了单次计算的速度和参数更新的频率，是目前深度学习最常用的优化方式。

6. 卷积神经网络(CNN)

- **为什么 CNN 适用于图像识别**：

- 一些模式比整张图片小得多；一个神经元不需要看到整个图像去发现模式，通过较少的参数连接到小区域（例如“喙”探测器识别出一张鸟的图片中的鸟嘴即可识别出鸟）。
- 同样的模式可能出现在图像的不同区域（如左上的鸟喙和中间的鸟喙），它们可以使用相同的一组参数（share 参数）。
- 对图像进行二次采样不会改变图像中的物体。

(1) 卷积神经网络中的层结构

- **五大基本组件**：
  - **卷积层 (Conv)**：利用卷积核提取局部特征。
    - **感受野与卷积核**：三个 3x3 卷积层（步长为1）的有效感受野等同于单个 7x7 卷积层。采用较小卷积核（如 3x3）能在相同感受野下减少参数、降低计算量并引入更多非线性，从而提升模型性能与泛化能力。
  - **池化层 (Pooling)**：通常使用最大池化，在特定窗口内提取最大值。作用是进行下采样以减少参数量，同时保留最显著特征。
  - **激活函数层 (Activation)**：引入非线性，使网络能学习复杂函数。
  - **归一化层 (Normalization)**：稳定数据分布，加速训练。
  - **全连接层 (FC)**：将池化层输出的多维特征图展平成一维向量，以便输入全连接层。最后计算全连接线性变换、Softmax 概率以及交叉熵损失。
- **Dropout**：
  - **核心含义**：在每次前向传播中，随机将部分神经元置零。即对每个神经元独立地以概率 p 进行保留（以概率 $1-p$ 将其置为 0）。丢失概率是超参数，通常取值为 0.5。
  - **优势与本质**：迫使网络采用冗余表示；阻碍特征间的协同适应。本质上是在训练一个共享参数的大型模型集合。
  - **训练阶段**：每次生成的 mask 是随机的（不保证恰好一半被保留），直接将被丢弃的神经元置零，不进行缩放。
  - **测试阶段**：所有神经元始终处于激活状态（不再丢弃）。为使输出强度与训练期望一致，我们必须调整激活值，使每个神经元满足：测试时输出 = 训练时预期输出（即将激活整体乘以保留率 p）。
- 卷积、池化后矩阵大小的计算公式：$O=\left\lfloor\frac{I+2P-K}{S}\right\rfloor+1$

  参数说明：
  - O：输出特征图的大小（Output size）
  - I：输入特征图的大小（Input size）
  - P：填充大小（Padding）
  - K：卷积核或池化核的大小（Kernel size）
  - S：步长（Stride）

(2) 激活函数

- **核心目标**：为模型引入非线性。
- **ReLU**：
  - 公式为 $a=\max(z,0)$ 或 $RELU(z)=\max(0,z)$。导数规则：在 $z>0$ 时导数为 1，在 $z\leq 0$ 时导数为 0。将所有负值过滤为 0。
  - 优势：在正区域内不饱和；计算效率极高；实际收敛速度远快于 Sigmoid；有效缓解梯度消失问题。
  - 缺陷：输出值非零中心化；当输入小于 0 时会出现死亡 ReLU 现象（即梯度为 0，参数停止更新）。Leaky ReLU 能更好解决此问题。
    - **Leaky ReLU**：ReLU 的变种。当 $z\leq 0$ 时给予一个极小的斜率（如 $0.01z$），避免神经元在负区间陷入“死亡”状态。

- **Sigmoid**：
  - 公式为 $a = \frac{1}{1+e^{-z}}$。导数恒等式为 $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$。
  - 优势：将数值（线性输出）压缩到 $[0,1]$ 区间，提供非线性表达能力，常用于二分类的输出层；输入为负时仍有梯度，可避免死亡现象。
  - 缺陷：导数最大值仅为 0.25。多层 Sigmoid 激活导致梯度逐渐衰减，极端 z 值导致梯度基本消失，靠近输入层的参数停止更新。
- **GELU**：
  - 公式为 $a = z \times \phi(z)$。
  - 优势：在 0 附近表现优异；平滑特性有助于实际训练过程。
  - 缺陷：计算成本高于 ReLU；大负值仍可能导致梯度趋近于零。

- **Softmax**
- 公式为 $P(Y=k|X=x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$。将模型输出的原始得分转换为概率分布。输出必须满足两个条件：所有输出为非负（即在 $[0,1]$ 区间内），并且总和为 1。
- 优势：将无界的原始得分转化为直观的概率分布；非常适合作为多分类任务的输出层，常与交叉熵损失函数结合使用。
- 缺陷：指数运算易放大数值差异，对极端异常值极度敏感，易导致数值溢出（上溢或下溢）；由于概率总和固定为 1，各类别间存在强互斥性，不适用于多标签分类任务。

(3) 卷积神经网络架构

- **VGGNet：小卷积核的胜利**
  - **核心改动**：抛弃大尺寸卷积核，全面采用堆叠的 3×3 **小卷积核**。
  - **优势**：
    1. **参数更少**：2个 3×3（感受野等同于 5×5）或 3个 3×3（感受野等同于 7×7）大大减少了参数量。
    2. **非线性更强**：每次小卷积后都接入 ReLU，多层堆叠带来了更多的非线性变换，显著增强了模型的特征表达能力。
- **ResNet：突破深度的极限**
  - **退化问题 (Degradation Problem)**：当普通网络（Plain Network）堆叠得过深时，训练误差不仅没有降低反而升高，信息在层层传递中丢失。
    - **残差连接 (Residual/Skip Connection)**：引入跨层的跳跃连接，公式为 $H(x) = F(x) + x$。
    - **作用机制**：如果新增的网络层不需要学习任何新特征，它只需将自身的权重推向 0，使 $F(x) = 0$，从而轻松实现**恒等映射 ($H(x) = x$)**。这保证了网络加深时性能绝不会比浅层网络差，成功训练出了上百层的超深网络。

(4) 权重初始化

- **全零初始化的灾难**：导致同一层所有神经元前向输出和反向梯度完全一致，陷入**对称性危机 (Symmetry Breaking)**，整层网络退化为单一神经元的表达能力。
- **极小/极大随机初始化**：分别会导致严重的**梯度消失**（激活值层层衰减趋零）和**梯度爆炸**（激活值层层放大至 NaN）。
- **经典初始化方法 (保持方差稳定)**：
  - **Xavier 初始化**：适合 Tanh / Sigmoid。
  - **Kaiming (He) 初始化**：专为 ReLU 设计。因 ReLU 截断负数导致方差减半，需额外乘 2 补偿能量损失。

(5) 数据预处理

- **核心操作**：对每个通道进行中心化和缩放，减去各通道均值，除以各通道标准差。
- **目的**：使不同维度的特征量级一致，让损失函数的地形更“圆”更均匀，从而防止震荡并加速收敛。

(6) 数据增强

- **手段**：随机裁剪、水平翻转、颜色抖动、平移缩放等。
- **目的**：在不增加标注成本的前提下人为扩充数据集，减少过拟合，显著提升模型的**泛化能力**。

(7) 迁移学习

- **解决痛点**：从头训练深层 CNN 需要海量数据，但在实际任务中往往面临“数据不足，极易过拟合”的困境。
- **核心思想**：利用在庞大数据集（如 ImageNet）上已经训练好的模型，将它们作为通用特征提取器应用到新任务上。
- **策略**：
  - **当目标数据集很小**：只修改最后一层（全连接层）以匹配新任务的类别数。**冻结 (Freeze)** 前面的所有卷积层权重，仅训练最后一层，防止过拟合。
  - **当目标数据集较大**：可以解冻更多后面的卷积层，甚至一起以极小的学习率进行微调，让模型更加适应新领域的数据特征。

(8) 超参数选择

- **科学调试步骤**：
  1. 检查初始 Loss。
  2. **小样本过拟合**：验证代码逻辑是否正确。
  3. 寻找使 Loss 下降的学习率（1e-1 到 1e-5 范围尝试）。
  4. 观察曲线：
     - **训练集与验证集差距巨大**：过拟合（需增加正则化或数据增强）。
     - **两线几乎重合且准确率低**：欠拟合（需增加模型容量或延长训练）。
- **学习率异常**：若 Loss 变为 **NaN**，通常是因为学习率设置得过大。

7. 归一化

- **层归一化(Layer Normalization)**：核心思想是学习能够对输入数据进行缩放/平移的参数。对每个样本（每一行）的所有特征独立进行标准化。
  1. 计算均值：$\mu_i = \frac{1}{d}\sum_{j=1}^{d} X_{i,j}$。
  2. 计算方差：$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}(X_{i,j}-\mu_i)^2$。
  3. 标准化：$\hat{X}_{i,j} = \frac{X_{i,j}-\mu_i}{\sqrt{\sigma_i^2}}$。
  4. 缩放和平移：$y = \gamma(x-\mu)/\sigma + \beta$ 或写为矩阵形式 $Y = \gamma \odot \hat{X} + \beta$。

一、KNN 分类计算题

编号	特征	类别
A	(1,2)	0
B	(2,3)	0
C	(3,3)	1
D	(5,2)	1
E	(4,0)	1

测试样本：$x = (3, 2)$, 使用 $K = 3$，欧氏距离

任务：

1. 计算每个点距离

$$d_{x,A} = 2 \quad d_{x,B} = \sqrt{2} \quad d_{x,C} = 1 \quad d_{x,D} = 2 \quad d_{x,E} = \sqrt{5}$$

2. 找出前 3 个近邻

前 3 个近邻为 C, B, A

3. 根据多数投票判断类别

类别为 1

4. 思考：若 $K = 5$ 呢？若 $K = 1$ 呢？

若 $K = 5$ 则类别为 1，若 $K = 1$ 则类别为 1

二、损失函数计算题

1. $Softmax$ + 交叉熵计算

输入：模型得分 $s = [2.0, 1.0, 0.1]$，标签为类别 0

任务：

$Softmax$ 输出

$$\hat{y} = \begin{bmatrix} \frac{e^2}{e^2+e^1+e^{0.1}} & \frac{e^1}{e^2+e^1+e^{0.1}} & \frac{e^{0.1}}{e^2+e^1+e^{0.1}} \end{bmatrix}^\top$$

交叉熵损失

$$L = -log(\frac{e^2}{e^2 + e^1 + e^{0.1}}) = log(e^2 + e^1 + e^{0.1}) - 2$$

思考：若标签改为类别 2，会发生什么变化？

$$L = -log(\frac{e^{0.1}}{e^2 + e^1 + e^{0.1}}) = log(e^2 + e^1 + e^{0.1}) - 0.1$$

损失会变大

2. $SVMhingeloss$ 计算

输入得分 $s = [3.0,\ 1.5,\ -0.5]$，标签为类别 0

任务：

写出每个 $margin$

$margin$ 计算公式如下：

$$margin_j = s_j - s_y + 1(j \neq y)$$

$margin$ 计算过程如下：

$$\begin{gathered} margin_0 = 0 \\ margin_1 = 1.5 - 3.0 + 1 = -0.5 \\ margin_2 = -0.5 - 3.0 + 1 = -2.5 \\ margin = [0,\ -0.5,\ -2.5] \end{gathered}$$

计算 SVM 损失

SVM 损失函数公式如下：

$$L_j = max(0,\ margin_j)$$

SVM 损失如下：

$$L = [0,\ 0,\ 0]$$

三、梯度下降与参数更新题

1. 单变量线性回归

输入 $x = 1$, 目标 $y = 4$，参数 $w_0 = 0, w_1 = 0$，使用 MSE 损失。

求预测值、损失值、梯度，并执行一轮参数更新（$\alpha = 0.1$）。

预测值如下：

$$\hat{y} = w_0 + w_1 x = 0 + 0 \times 1 = 0$$

损失值如下：

$$L = \frac{1}{2}\ (\hat{y} - y)^2 = \frac{1}{2}\ (0 - 4)^2 = 8$$

梯度计算如下：

$$\begin{gathered} \frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \hat{y}}\ \cdot\ \frac{\partial \hat{y}}{\partial w_0} = (\hat{y} - y)\ \cdot\ 1 = (-4)\ \cdot\ 1 = -4 \\ \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}}\ \cdot\ \frac{\partial \hat{y}}{\partial w_1} = (\hat{y} - y)\ \cdot\ x = (-4)\ \cdot\ 1 = -4 \end{gathered}$$

梯度向量如下：

$$\nabla L = [-4, -4]$$

参数更新过程如下：

$$w_{0(new)} = w_{0(old)} - \alpha \cdot \frac{\partial L}{\partial w_0} = 0 - 0.1 \cdot (-4) = 0.4$$
$$w_{1(new)} = w_{1(old)} - \alpha \cdot \frac{\partial L}{\partial w_1} = 0 - 0.1 \cdot (-4) = 0.4$$

新的参数如下：

$$w_0 = 0.4, \quad w_1 = 0.4$$

2. 多变量线性模型

$x = [2, 3], w_0 = 0, w_1 = 1, w_2 = -1, y = 1$

使用 MSE，求梯度向量和新的参数值（$\alpha = 0.05$）。

预测值如下：

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 = 0 + 1 \cdot 2 + (-1) \cdot 3 = -1$$

损失值如下：

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(-1 - 1)^2 = 2$$

梯度计算如下：

$$\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_0} = (\hat{y} - y) \cdot 1 = (-2) \cdot 1 = -2$$
$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1} = (\hat{y} - y) \cdot x_1 = (-2) \cdot 2 = -4$$
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} = (\hat{y} - y) \cdot x_2 = (-2) \cdot 3 = -6$$

梯度向量如下：

$$\nabla L = [-2, -4, -6]$$

参数更新过程如下：

$$w_{0(new)} = w_{0(old)} - \alpha \cdot \frac{\partial L}{\partial w_0} = 0 - 0.05 \cdot (-2) = 0.1$$
$$w_{1(new)} = w_{1(old)} - \alpha \cdot \frac{\partial L}{\partial w_1} = 1 - 0.05 \cdot (-4) = 1.2$$
$$w_{2(new)} = w_{2(old)} - \alpha \cdot \frac{\partial L}{\partial w_2} = -1 - 0.05 \cdot (-6) = -0.7$$

新的参数如下：

$$w_0 = 0.1, \quad w_1 = 1.2, \quad w_2 = -0.7$$

3. 正则化

题 2 的基础上，若损失函数增加了 $L2$ 正则化项，正则化系数 $\lambda = 0.1$，那么损失函数值为多少，梯度向量又是多少？

$L2$ 正则化损失如下：

$$L = L(W) + \frac{\lambda}{2}(w_1^2 + w_2^2) = 2 + 0.1 = 2.1$$

梯度计算如下：

$$\frac{\partial L}{\partial w_0} = -2$$
$$\frac{\partial L}{\partial w_1} = -4 + \lambda w_1 = -4 + 0.1 \times 1 = -3.9$$
$$\frac{\partial L}{\partial w_2} = -6 + \lambda w_2 = -6 + 0.1 \times (-1) = -6.1$$

梯度向量如下：

$$\nabla L = [-2, -3.9, -6.1]$$

参数更新过程如下：

$$w_{0(new)} = w_{0(old)} - \alpha \cdot \frac{\partial L}{\partial w_0} = 0 - 0.05 \cdot (-2) = 0.1$$
$$w_{1(new)} = w_{1(old)} - \alpha \cdot \frac{\partial L}{\partial w_1} = 1 - 0.05 \cdot (-3.9) = 1.195$$
$$w_{2(new)} = w_{2(old)} - \alpha \cdot \frac{\partial L}{\partial w_2} = -1 - 0.05 \cdot (-6.1) = -0.695$$

新的参数如下：

$$w_0 = 0.1, \quad w_1 = 1.195, \quad w_2 = -0.695$$

四、反向传播计算题

1. 单层神经元

输入 $x = [1, 2]$，权重 $W = [0.5, -0.5]$，激活函数为 $sigmoid$，输出 y，目标值为 $y = 1$。损失为 MSE。

请计算损失 L，并求 $\frac{\partial L}{\partial W}$

线性部分输出如下：

$$z = W_1 x_1 + W_2 x_2 = 0.5 \cdot 1 + (-0.5) \cdot 2 = -0.5$$

激活输出如下：

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{0.5}}$$

损失 L 输出如下：

$$L = \frac{1}{2}\ (\hat{y} - y)^2 = \frac{1}{2}\ (\frac{1}{1 + e^{0.5}} - 1)^2 = \frac{e}{2(1 + e^{0.5})^2}$$

梯度计算如下：

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = (\hat{y} - y) \cdot \frac{e^{-z}}{(1+e^{-z})^2} \cdot x_1 = -\frac{e^{0.5}}{1+e^{0.5}} \cdot \frac{e^{0.5}}{(1+e^{0.5})^2} \cdot 1 = -\frac{e}{(1+e^{0.5})^3}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_2} = (\hat{y} - y) \cdot \frac{e^{-z}}{(1+e^{-z})^2} \cdot x_2 = -\frac{e^{0.5}}{1+e^{0.5}} \cdot \frac{e^{0.5}}{(1+e^{0.5})^2} \cdot 2 = -\frac{2e}{(1+e^{0.5})^3}$$

梯度向量如下：

$$\frac{\partial L}{\partial W} = \nabla L = [-\frac{e}{(1+e^{0.5})^3}, -\frac{2e}{(1+e^{0.5})^3}]$$

一、矩阵乘法+ $ReLU$ 激活+ 反向传播计算题

已知输入矩阵：

$$X = \begin{bmatrix} 1 & 0 & -1 \\ 2 & -1 & 3 \\ 0 & 1 & 2 \end{bmatrix}$$

参数矩阵与偏置：

$$W = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & -2 \end{bmatrix}$$

前向传播步骤：

$$Z = W\,X, \qquad A = RELU(Z)$$

其中 $RELU$ 激活函数为：

$$RELU(z) = max(0, z)$$

问题1：前向计算

1. 计算线性变换输出矩阵 Z

$$Z = W\,X = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & -2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \\ 2 & -1 & 3 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & -5 \\ 7 & -5 & 4 \end{bmatrix}$$

2. 计算激活输出矩阵 A

$$A = RELU(Z) = \begin{bmatrix} 0 & 1 & 0 \\ 7 & 0 & 4 \end{bmatrix}$$

问题2：反向传播计算

(1) 利用 $RELU$ 的导数（在 $z > 0$ 时为 1，在 $z \leq 0$ 时为 0）计算 $\frac{\partial L}{\partial Z}$

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \frac{\partial A}{\partial Z} = \begin{bmatrix} 1 & 0 & -2 \\ -1 & 2 & 1 \end{bmatrix} \odot \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

(2) 推导并计算 $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z}\,X^\top, \qquad \frac{\partial L}{\partial X} = W^\top\,\frac{\partial L}{\partial Z}$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z}\,X^\top = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 0 \\ 0 & -1 & 1 \\ -1 & 3 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ -2 & 1 & 2 \end{bmatrix}$$

$$\frac{\partial L}{\partial X} = W^\top\,\frac{\partial L}{\partial Z} = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 0 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ -3 & 0 & 3 \\ 2 & 0 & -2 \end{bmatrix}$$

问题3：梯度下降更新

沿用第 2 题得到的梯度，设学习率 $\eta = 0.1$，使用标准梯度下降更新第 1 题中的参数：

$$W^{(new)} = W^{(old)} - \eta\,\frac{\partial L}{\partial W}$$

(1) 计算 $W^{(new)}$ 的数值结果

$$W^{(new)} = W^{(old)} - \eta\,\frac{\partial L}{\partial W} = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & -2 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ -0.2 & 0.1 & 0.2 \end{bmatrix} = \begin{bmatrix} 2 & -1 & 0 \\ 1.2 & 2.9 & -2.2 \end{bmatrix}$$

二、卷积神经网络前向传播计算题

网络结构如下：

- 输入图像：4×4 RGB 图像（3 通道）
- 卷积层：一个 $2 \times 2 \times 3$ 卷积核，步长为 2，补全为 1，激活函数为 $ReLU$
- 池化层：2×2 最大池化，步长为 2

- 全连接层：展平后连接 3 个神经元，使用 $softmax$ 输出
- 损失函数为交叉熵损失 + $L1$ 正则化（不含偏置）

图像输入如下：

$$\mathbf{R}\ \textbf{通道}:\begin{bmatrix}1&2&0&1\\0&1&2&0\\1&0&1&2\\2&1&0&1\end{bmatrix}\quad \mathbf{G}\ \textbf{通道}:\begin{bmatrix}0&1&0&1\\2&1&1&0\\1&2&0&1\\1&1&2&0\end{bmatrix}\quad \mathbf{B}\ \textbf{通道}:\begin{bmatrix}1&0&2&1\\1&0&1&1\\2&1&0&2\\0&1&1&0\end{bmatrix}$$

卷积核：

$$W^{(R)}=\begin{bmatrix}1&0\\-1&1\end{bmatrix},\quad W^{(G)}=\begin{bmatrix}0&1\\1&-1\end{bmatrix},\quad W^{(B)}=\begin{bmatrix}1&-1\\0&1\end{bmatrix}$$

全连接层参数：

$$W_{fc}=\begin{bmatrix}1&0&-1&1\\0&1&1&-1\\1&-1&0&0\end{bmatrix},\quad b_{fc}=\begin{bmatrix}0\\1\\-1\end{bmatrix}$$

真实标签：

$$y=\begin{bmatrix}1\\0\\0\end{bmatrix}$$

即正确类别为第一个

正则化系数：$\lambda=0.01$

要求：

(1) 卷积层前向传播：

对图像进行三通道卷积，三通道分别进行卷积操作后相加，再应用 $ReLU$，写出输出特征图

分别对 R/G/B 三个通道做 2×2 卷积（stride=2,pad=1），得到三个 3×3 特征图，再相加：

$$Y_R=\begin{bmatrix}1&-2&-1\\1&2&-2\\0&1&1\end{bmatrix},\quad Y_G=\begin{bmatrix}0&1&1\\1&3&1\\1&2&0\end{bmatrix},\quad Y_B=\begin{bmatrix}1&2&0\\1&-1&1\\0&0&0\end{bmatrix}$$

三通道相加：

$$Y=Y_R\ +\ Y_G\ +\ Y_B=\begin{bmatrix}2&1&0\\3&4&0\\1&3&1\end{bmatrix}$$

$ReLU$ 激活：

$$A_{conv}=ReLU(A)=\begin{bmatrix}2&1&0\\3&4&0\\1&3&1\end{bmatrix}$$

(2) 池化层前向传播：

对特征图进行 2×2 最大池化，步长为 1，写出池化层输出

池化层输出为：

$$A_{pool}=\begin{bmatrix}4&4\\4&4\end{bmatrix}$$

(3) 输出层与损失计算：

将池化层输出展平成向量 $x=[a1,a2,a3,a4]^T$，计算输出层线性变换 $z=W_{fc}\cdot x+b_{fc}$

池化层输出展平为：

$$x=[4\quad 4\quad 4\quad 4]^T$$

全连接层输出为

$$z = W_{fc} \cdot x + b_{fc} = \begin{bmatrix} 1 & 0 & -1 & 1 \\ 0 & 1 & 1 & -1 \\ 1 & -1 & 0 & 0 \end{bmatrix} \cdot [4 \quad 4 \quad 4 \quad 4]^T + \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ -1 \end{bmatrix}$$

使用 $softmax$ 计算 $\hat{y} = \frac{e^{s_i}}{\sum_{j=1}^{3} e^{s_j}} \quad (i = 1, 2, 3)$

最终经过激活函数 $softmax$ 输出为：

$$\hat{y} = \frac{e^{z_i}}{e^4 + e^5 + e^{-1}} = \begin{bmatrix} \frac{e^4}{e^4+e^5+e^{-1}} \\ \frac{e^5}{e^4+e^5+e^{-1}} \\ \frac{e^{-1}}{e^4+e^5+e^{-1}} \end{bmatrix}$$

计算交叉熵损失 $L_{ce} = -\sum_{i=1}^{3} y_i log(\hat{y}_i)$

交叉熵损失如下：

$$L_{ce} = -\sum_{i=1}^{3} y_i log(\hat{y}_i) = -[1 \cdot log(\frac{e^4}{e^4 + e^5 + e^{-1}}) + 0 \cdot log(\frac{e^5}{e^4 + e^5 + e^{-1}}) + 0 \cdot log(\frac{e^{-1}}{e^4 + e^5 + e^{-1}})] = -log(\frac{e^4}{e^4 + e^5 + e^{-1}})$$

计算 $L1$ 正则化项（对 $W^{(R)}$，$W^{(G)}$，$W^{(B)}$，W_{fc}） $L_{reg} = \lambda \sum_{i,j} |W_{fc_{i,j}}|$

$$\|W^{(R)}\|_1 = 3, \quad \|W^{(G)}\|_1 = 3, \quad \|W^{(B)}\|_1 = 3, \quad \|W_{(fc)}\|_1 = 8$$

$$L_{reg} = 0.01 \times (3 + 3 + 3 + 8) = 0.17$$

写出总损失 $L = L_{ce} + L_{reg}$

$$L = -log(\frac{e^4}{e^4 + e^5 + e^{-1}}) + 0.17$$

三、Layer Normalization 计算题（两行样本，含公式）

某层的输出为：

$$X = \begin{bmatrix} 2 & 4 & 6 & 8 \\ 1 & 3 & 5 & 7 \end{bmatrix}$$

其中每一行代表一个样本，特征维度为 4。

Layer Normalization 对每一个样本（每一行）进行标准化，其计算步骤如下：

$$\mu_i = \frac{1}{d} \sum_{j=1}^{d} X_{i,j}, \quad \sigma_i^2 = \frac{1}{d} \sum_{j=1}^{d} (X_{i,j} - \mu_i)^2$$

标准化：

$$\hat{X}_{i,j} = \frac{X_{i,j} - \mu_i}{\sqrt{\sigma_i^2}}$$

缩放和平移（本题中为恒等映射）：

$$Y = \gamma \odot \hat{X} + \beta, \quad \gamma = [1 \quad 1 \quad 1 \quad 1], \quad \beta = [0 \quad 0 \quad 0 \quad 0]$$

请计算：

(1) 两个样本的均值 μ_1, μ_2

$$\mu_1 = \frac{2 + 4 + 6 + 8}{4} = 5$$
$$\mu_2 = \frac{1 + 3 + 5 + 7}{4} = 4$$

(2) 两个样本的方差 σ_1^2, σ_2^2

$$\sigma_1^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5$$
$$\sigma_2^2 = \frac{(1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2}{4} = 5$$

(3) 标准化结果矩阵 $\hat{X}$

$$\hat{X} = \begin{bmatrix} -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{3}{\sqrt{5}} \\ -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{3}{\sqrt{5}} \end{bmatrix}$$

(4) LayerNorm 的最终输出矩阵 Y

$$Y = \gamma \odot \hat{X} + \beta = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \odot \begin{bmatrix} -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{3}{\sqrt{5}} \\ -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{3}{\sqrt{5}} \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{3}{\sqrt{5}} \\ -\frac{3}{\sqrt{5}} & -\frac{1}{\sqrt{5}} & \frac{1}{\sqrt{5}} & \frac{3}{\sqrt{5}} \end{bmatrix}$$

四、$Dropout$ 计算题（$VanillaDropout$）

已知某一层的输出矩阵为：

$$H = \begin{bmatrix} 4 & -2 & 0 & 6 \\ 1 & 3 & -4 & 2 \end{bmatrix}$$

$Dropout$ 保留率为：

$$p = 0.5$$

说明：$Dropout$ 的含义是对每个神经元独立地以概率 p 进行保留，以概率 $1-p$ 将其置为 0。因此，每次得到的 $mask$ 是随机的，并不保证恰好一半神经元被保留。

假设某次训练中生成的 $Dropoutmask$ 为：

$$mask = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}$$

在 $VanillaDropout$ 中：

- 训练阶段：直接将被丢弃的神经元置零，不进行缩放

$$H_{train} = H \odot mask$$

- 测试阶段：不再丢弃神经元，为使输出强度与训练期望一致，需要将激活整体乘以保留率

$$H_{test} = p \cdot H$$

问题：

计算训练阶段输出 H_{train}

$$H_{train} = H \odot mask = \begin{bmatrix} 4 & -2 & 0 & 6 \\ 1 & 3 & -4 & 2 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 3 & -4 & 2 \end{bmatrix}$$

计算测试阶段输出 H_{test}

$$H_{test} = p \cdot H = 0.5 \cdot \begin{bmatrix} 4 & -2 & 0 & 6 \\ 1 & 3 & -4 & 2 \end{bmatrix} = \begin{bmatrix} 2 & -1 & 0 & 3 \\ 0.5 & 1.5 & -2 & 1 \end{bmatrix}$$

简要解释测试阶段为什么要乘以p（一句话即可）

保证测试阶段的输出强度与训练期望一致。

HW 1：线性分类器计算题（三分类）

已知条件：

这是一个**三分类**线性分类问题，类别集合为 $\{0:\text{猫}, 1:\text{狗}, 2:\text{兔子}\}$。

- **模型公式**：$s = Wx + b$
- **参数设定**：
  - 权重矩阵 $W = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -1 & 0 \end{bmatrix}$
  - 偏置向量 $b = \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix}$
- **训练样本**：
  - **样本 1**：$x_1 = [2, -1]^T$，标签：猫（索引 0）
  - **样本 2**：$x_2 = [0, 2]^T$，标签：狗（索引 1）
  - **样本 3**：$x_3 = [-2, 0]^T$，标签：兔子（索引 2）

---

1 得分向量计算与预测

根据公式 $s = Wx + b$ 计算各样本得分：

- **样本 1**：
  - $s_1 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} 2+1 \\ 0-2 \\ -2+0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} 3.5 \\ -2.5 \\ -2 \end{bmatrix}$
  - **预测类别**：猫（得分最高项索引为 0）
- **样本 2**：
  - $s_2 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 4 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 3.5 \\ 0 \end{bmatrix}$
  - **预测类别**：狗（得分最高项索引为 1）
- **样本 3**：
  - $s_3 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} -2 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -0.5 \\ 2 \end{bmatrix}$
  - **预测类别**：兔子（得分最高项索引为 2）

---

2 多分类 SVM 损失计算（Hinge Loss）

损失函数定义：$l_{SVM}(s, y) = \sum_{j \neq y} \max(0, s_j - s_y + 1)$

- **样本 1 损失（L_1）**：
  - $\max(0, -2.5 - 3.5 + 1) + \max(0, -2 - 3.5 + 1) = 0 + 0 = 0$
- **样本 2 损失（L_2）**：
  - $\max(0, -1.5 - 3.5 + 1) + \max(0, 0 - 3.5 + 1) = 0 + 0 = 0$
- **样本 3 损失（L_3）**：
  - $\max(0, -1.5 - 2 + 1) + \max(0, -0.5 - 2 + 1) = 0 + 0 = 0$
- **平均 SVM 损失**：$\bar{L}_{SVM} = \frac{1}{3}(0 + 0 + 0) = 0$

---

3 交叉熵损失计算（Cross-Entropy Loss）

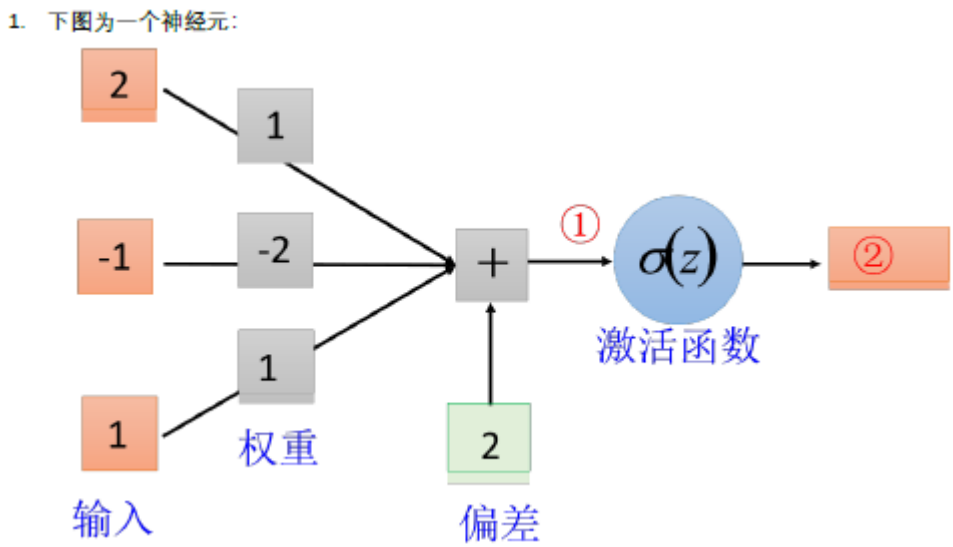
损失函数定义：$l_{CE}(s, y) = \log \frac{\sum e^{s_j}}{e^{s_y}} = \log\left(\sum e^{s_j - s_y}\right)$

- **样本 1 损失（$L_{CE,1}$）**：
  - $\log\left(e^{3.5-3.5} + e^{-2.5-3.5} + e^{-2-3.5}\right) = \log\left(1 + e^{-6} + e^{-5.5}\right)$
- **样本 2 损失（$L_{CE,2}$）**：
  - $\log\left(e^{-1.5-3.5} + e^{3.5-3.5} + e^{0-3.5}\right) = \log\left(1 + e^{-5} + e^{-3.5}\right)$
- **样本 3 损失（$L_{CE,3}$）**：
  - $\log\left(e^{-1.5-2} + e^{-0.5-2} + e^{2-2}\right) = \log\left(1 + e^{-3.5} + e^{-2.5}\right)$
- **平均交叉熵损失**：
  - $\bar{L}_{CE} = \frac{1}{3}\left[\log\left(1 + e^{-6} + e^{-5.5}\right) + \log\left(1 + e^{-5} + e^{-3.5}\right) + \log\left(1 + e^{-3.5} + e^{-2.5}\right)\right]$

HW 2：神经元反向传播计算

已知条件：

- 输入向量：$x = [2, -1, 1]$
- 权重向量：$w = [1, -2, 1]$
- 偏差：$b = 2$
- 下图是该神经元：



1）若激活函数为 Sigmoid 函数，画出计算图，使用反向传播计算权重和偏差的梯度。

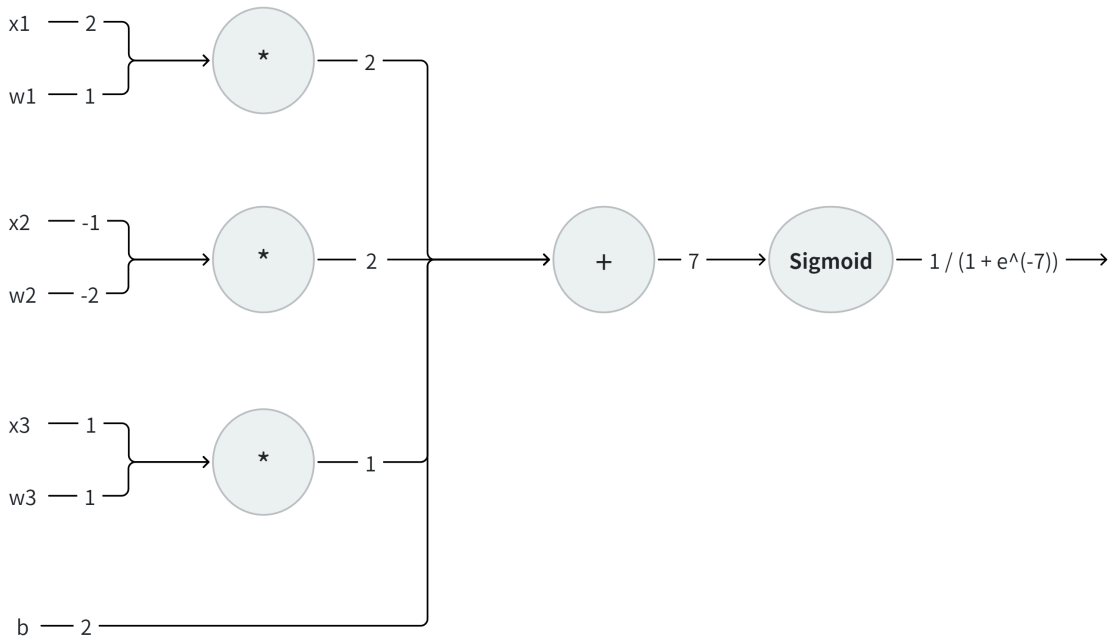
2）若激活函数为 ReLU 函数，画出计算图，使用反向传播计算权重和偏差的梯度。

前向传播计算（线性输出 z）：

- $z = (w_1 \times x_1) + (w_2 \times x_2) + (w_3 \times x_3) + b$
- $z = (1 \times 2) + (-2 \times -1) + (1 \times 1) + 2 = 7$

1 情况一：Sigmoid 激活函数

- **激活函数公式**：$\sigma(z) = \frac{1}{1+e^{-z}}$
- **计算图**：

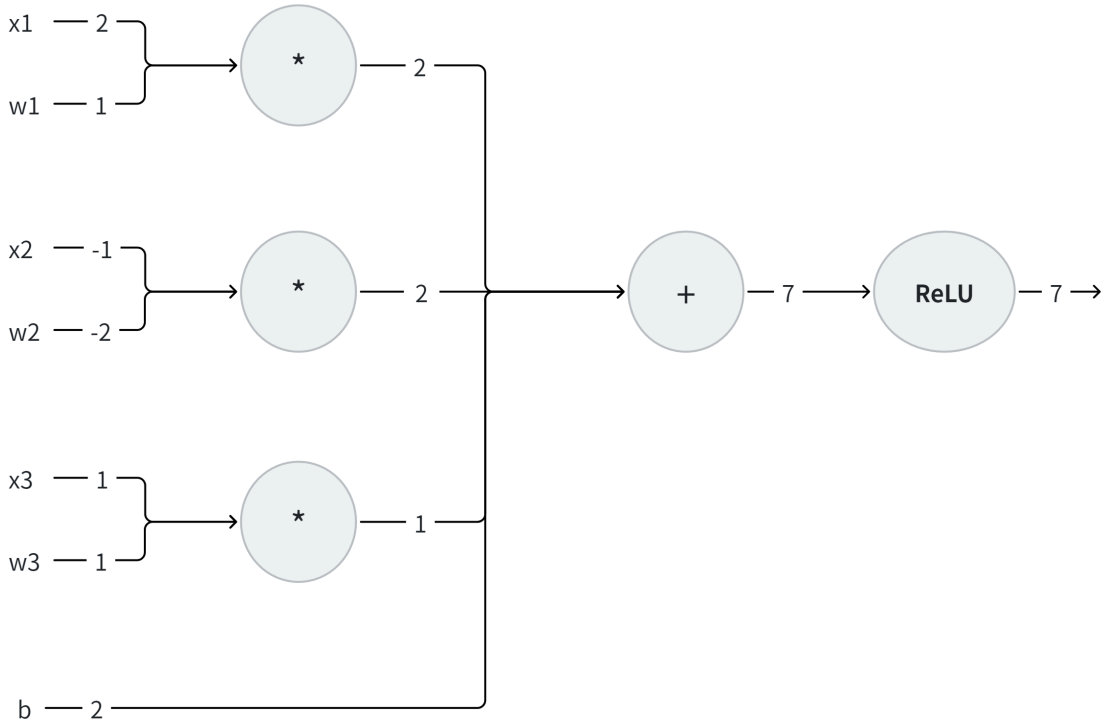


- **局部梯度（对 z 求导）**：在 $z = 7$ 处，$\frac{\partial \sigma(z)}{\partial z} = \frac{e^{-7}}{(1+e^{-7})^2}$
- **线性层梯度**：
  - $\frac{\partial z}{\partial w_1} = x_1 = 2$
  - $\frac{\partial z}{\partial w_2} = x_2 = -1$
  - $\frac{\partial z}{\partial w_3} = x_3 = 1$
  - $\frac{\partial z}{\partial b} = 1$
- **最终链式求导结果（反向传播计算权重和偏差的梯度）**：
  - $\frac{\partial \sigma(z)}{\partial w_1} = \frac{\partial \sigma(z)}{\partial z} \times \frac{\partial z}{\partial w_1} = \frac{2 \times e^{-7}}{(1+e^{-7})^2}$
  - $\frac{\partial \sigma(z)}{\partial w_2} = \frac{\partial \sigma(z)}{\partial z} \times \frac{\partial z}{\partial w_2} = \frac{-e^{-7}}{(1+e^{-7})^2}$
  - $\frac{\partial \sigma(z)}{\partial w_3} = \frac{\partial \sigma(z)}{\partial z} \times \frac{\partial z}{\partial w_3} = \frac{e^{-7}}{(1+e^{-7})^2}$
  - $\frac{\partial \sigma(z)}{\partial b} = \frac{\partial \sigma(z)}{\partial z} \times \frac{\partial z}{\partial b} = \frac{e^{-7}}{(1+e^{-7})^2}$

---

2 情况二：ReLU 激活函数

- **激活函数公式**：$ReLU(z) = \max(0, z)$
- **计算图**：



- **局部梯度（对 z 求导）**：因为 $z = 7 > 0$，所以 $\frac{\partial \sigma(z)}{\partial z} = 1$
- **线性层梯度**：与情况一完全相同（$\frac{\partial z}{\partial w_1} = 2, \frac{\partial z}{\partial w_2} = -1, \frac{\partial z}{\partial w_3} = 1, \frac{\partial z}{\partial b} = 1$）
- **最终链式求导结果（反向传播计算权重和偏差的梯度）**：
  - $\frac{\partial \sigma(z)}{\partial w_1} = 1 \times 2 = 2$
  - $\frac{\partial \sigma(z)}{\partial w_2} = 1 \times (-1) = -1$
  - $\frac{\partial \sigma(z)}{\partial w_3} = 1 \times 1 = 1$
  - $\frac{\partial \sigma(z)}{\partial b} = 1 \times 1 = 1$

HW 3：矩阵乘法与 ReLU 激活的反向传播

已知条件：

- 输入矩阵 $X = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$
- 权重矩阵 $W = \begin{bmatrix} 1 & -2 \\ -1 & 3 \end{bmatrix}$
- 上游梯度 $\frac{\partial L}{\partial Y} = \begin{bmatrix} 1 & -1 \\ 0.5 & 2 \end{bmatrix}$

---

1 题目 1：前向传播

- **计算线性输出 Z**（$Z = XW$）：
  $$Z = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} 2 & -5 \\ 2 & -4 \end{bmatrix}$$
- **计算激活输出 Y**（$Y = ReLU(Z)$）：
  $$Y = \max(0, Z) = \begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix}$$

---

2 题目 2：ReLU 层反向传播

- **局部梯度 $\frac{\partial Y}{\partial Z}$**（根据 ReLU 规则，大于 0 为 1，小于等于 0 为 0）：
  $$\frac{\partial Y}{\partial Z} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$
- **计算对 Z 的梯度 $\frac{\partial L}{\partial Z}$**（利用链式法则进行逐元素相乘 $\odot$）：
  $$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial Y} \odot \frac{\partial Y}{\partial Z} = \begin{bmatrix} 1 & -1 \\ 0.5 & 2 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0.5 & 0 \end{bmatrix}$$

---

3 题目 3：矩阵乘法层反向传播

- **计算对权重 W 的梯度**（$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Z}$）：
  - $X^T = \begin{bmatrix} 1 & 2 \\ -1 & 0 \end{bmatrix}$
  - $\frac{\partial L}{\partial W} = \begin{bmatrix} 1 & 2 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ -1 & 0 \end{bmatrix}$
- **计算对输入 X 的梯度**（$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^T$）：
  - $W^T = \begin{bmatrix} 1 & -1 \\ -2 & 3 \end{bmatrix}$
  - $\frac{\partial L}{\partial X} = \begin{bmatrix} 1 & 0 \\ 0.5 & 0 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 0.5 & -0.5 \end{bmatrix}$

---

4 题目 4：思考题

1. **当 $Z_{ij} < 0$ 时，对应的 $\frac{\partial L}{\partial Z_{ij}}$ 会怎样？**
   - 答：会变成 0。因为 ReLU 函数在输入小于 0 时的导数为 0，导致回传的梯度被直接阻断。
2. **如果 Z 的大部分元素小于 0，会出现什么问题？**
   - 答：会导致大部分梯度无法有效回传。在反向传播更新时，这部分权重 W 的梯度也会变成 0，导致参数停止更新，失去对数据的响应能力。这种现象在深度学习中被称为**神经元“死亡”（死亡 ReLU）**。
3. **为什么要先求 $\frac{\partial L}{\partial Z}$，再求 $\frac{\partial L}{\partial W}$ 和 $\frac{\partial L}{\partial X}$？**
   - 答：这是由微积分中的**链式法则**决定的。在反向传播中，梯度必须从输出层向输入层逐层剥开计算。参数 W 和 X 只能通过中间变量 Z 影响最终结果，因此必须先求出对激活前变量 Z 的梯度，才能以此为基础继续向后回传。

CNN 练习题：CIFAR-10 特征图维度计算

基本参数：

- 分类目标：10 个类别
- 图像输入维度：宽 32、高 32、通道 3
- 空间尺寸推导公式：$Output = \frac{Input - Kernel + 2 \times Padding}{Stride} + 1$

---

1 卷积与池化操作的特征图形状

- **第一层卷积（Conv1）**：
  - 设定：包含 32 个滤波器，尺寸为 3×3×3；步幅为 1；边缘填充为 1。
  - 空间尺寸计算：$W = \frac{32-3+2\times1}{1} + 1 = 32$
  - 最终形状：32×32×32
- **第二层卷积（Conv2）**：
  - 设定：包含 32 个滤波器，尺寸为 3×3×32；步幅为 1；无边缘填充（0）。
  - 空间尺寸计算：$W = \frac{32-3+2\times0}{1} + 1 = 30$
  - 最终形状：30×30×32
- **第一层池化（Pool1）**：
  - 设定：池化核尺寸为 2×2；步幅为 2；无边缘填充（0）。
  - 空间尺寸计算：$W = \frac{30-2+2\times0}{2} + 1 = 15$
  - 最终形状：15×15×32
- **第三层卷积（Conv3）**：
  - 设定：包含 64 个滤波器，尺寸为 3×3×32；步幅为 1；边缘填充为 1。
  - 空间尺寸计算：$W = \frac{15-3+2\times1}{1} + 1 = 15$
  - 最终形状：15×15×64
- **第四层卷积（Conv4）**：
  - 设定：包含 64 个滤波器，尺寸为 3×3×64；步幅为 1；无边缘填充（0）。
  - 空间尺寸计算：$W = \frac{15-3+2\times0}{1} + 1 = 13$
  - 最终形状：13×13×64
- **第二层池化（Pool2）**：
  - 设定：池化核尺寸为 3×3；步幅为 2；无边缘填充（0）。
  - 空间尺寸计算：$W = \frac{13-3+2\times0}{2} + 1 = 6$
  - 最终形状：6×6×64

2 全连接层与输出端维度

- **全连接层的输入特征向量维度**：
  - 解析：需将 Pool2 输出的三维特征矩阵展平，即 6 × 6 × 64 = 2304。
  - 结果：(2304,)
- **网络最终输出层的维度**：
  - 解析：由 CIFAR-10 数据集的总类别数决定。 结果：(10,)

数据分析知识点复习

一、 语法

1. PEP8 编码规范与对象模型

- **代码注释**：单行注释使用 `#`；多行注释使用三对双引号 `"""` 或单引号 `'''` 包裹。
- **命名规范**：
  - **全局变量**：全大写加下划线（如 `MAX_VALUE`）。
  - **私有变量**：小写字母加单前导下划线（如 `_private_var`）。
  - **内置/魔术变量**：双前导和双后置下划线（如 `__init__`）。
  - **类名**：首字母大写/驼峰式（如 `ClassName`）。
- **空格规范**：
  - 赋值/比较/逻辑运算符（`=`, `==`, `and` 等）前后**加**空格。
  - 高优先级运算符（`*`, `/`, `**`）前后**不加**空格。
- **对象三要素**：
  - **身份 (Identity)**：内存地址标识，用 `id()` 获取，产生后不可改变。
  - **类型 (Type)**：数据分类，用 `type()` 获取。
  - **值 (Value)**：存储的具体数据。
- **连环赋值**：`a = b = c = 1` 会将三个变量绑定到同一内存地址。

2. 核心数据类型

- **整型 (int)**：支持无限精度，大小仅受限于运行环境内存。十六进制以 `0x` 开头，八进制以 `0o` 开头。空参 `int()` 默认返回 `0`。
- **浮点型 (float)**：遵循 IEEE754 双精度标准，独占 8 个字节内存。
- **复数 (complex)**：由实部和虚部（以 `j` 或 `J` 结尾）组成。**注意**：复数只能比较是否相等（`==`），绝对不能比较大小（`>` 或 `<`）。
- **布尔型 (bool)**：
  - 是整型的子类（`False` 等效于 `0`，`True` 等效于 `1`）。
  - **隐式假值**：数值 `0`、`None`、空字符串 `""`、空列表 `[]`、空元组 `()`。
  - **陷阱**：`bool("False")` 结果为 `True`（因为是非空字符串）；`bool([0])` 结果为 `True`（因为是非空列表）。

3. 运算符与优先级

- **算术运算符**：
  - `//`：取整除（向负无穷取整数商）。
  - `%`：取模（求余数）。
  - `**`：幂运算。具有**右结合性**（如 `2 ** 3 ** 2` 等价于 `2 ** 9`），且优先级高于单目负号（如 `-3 ** 2` 等价于 `-(3 ** 2)`，结果为 `-9`）。
- **逻辑运算符**：
  - 优先级自高到低：`not` > `and` > `or`。
  - **短路机制**：如 `False and (10/0)` 会直接返回 `False`，不会引发除零错误。
- **链式比较**：`1 < 2 == 2` 是合法的，等效于 `(1 < 2) and (2 == 2)`。

二、 流程控制

- **代码块边界**：严格依赖**行首缩进**来划分代码块，不使用大括号。
- **条件分支 (`if-elif-else`)**：
  - 只要触发匹配项就会跳出整个结构（排他性）。
  - 若直接并列多个独立的 `if`，则每个条件都会被强制判定一次。
- **循环遍历 (`for` / `while`)**：
  - `range(start, stop, step)`：遵循**左闭右开**原则，永远取不到 `stop` 的边界值。
- **循环控制字**：
  - `break`：立刻终结并跳出**当前这一层**循环。
  - `continue`：中断**本次**迭代的剩余代码，直接进入下一轮条件判定。
- **循环 `else` 分支**：
  - **触发条件**：只有当循环**正常执行完毕**时才会执行。
  - **失效条件**：一旦循环体内部触发了 `break`，`else` 代码块将被直接抛弃，绝对不会执行。

三、 组合数据类型

- 常见的不可变对象
  - **数字类型**：`int`，`float`，`bool`，`complex`
  - **字符串**：`str`
  - **元组**：`tuple`
  - **冻结集合**：`frozenset`
  - **字节串**：`bytes`
- 常见的可变对象
  - **列表**：`list`
  - **字典**：`dict`
  - **集合**：`set`
  - **字节数组**：`bytearray`
  - 用户自定义的类实例

Python 提供了四种主要的组合数据类型，按**是否可变**和**是否有序**区分：

1. 列表 (List) - `[]`

- **特点**：有序、可变、可变长、允许重复、支持异构嵌套。
- **访问**：通过索引访问（如 `list[0]`，`list[-1]`），支持切片（`list[start:end:step]`）。
- **常用方法**：
  - 增加：`append(x)`（末尾追加）、`insert(i, x)`（定点插入）、`extend(seq)`（合并扩展）。
  - 删除：`del list[i]`、`remove(x)`（删除首个匹配值）、`pop(i)`（弹出元素，默认末尾）、`clear()`。
  - 其他：`reverse()`（反转）、`sort(key=None, reverse=False)`（排序）、`max()`（最大值）、`min()`（最小值）、`copy()`（复制列表）、`count(x)`（统计该元素在列表中出现的次数）。

2. 元组 (Tuple) - `()`

- **特点**：有序、**不可变**（防弹衣版列表）、可变长、允许重复、支持异构嵌套。
- **定义陷阱**：单元素元组必须加逗号，即 `(1,)`，否则括号会被视为算术运算符。
- **操作**：支持索引和切片，不支持修改和删除内部元素，但可用 `del tuple` 删除整个元组对象。

3. 字典 (Dictionary) - `{key: value}`

- **特点**：无序、可变、**键唯一且不可变**（如数字、字符串、元组）。
- **操作**：
  - 读取/修改：`dict[key]` 或 `dict.get(key)`。
  - 删除：`del dict[key]`、`pop(key)`、`clear()`。
  - 视图：`keys()`、`values()`、`items()`。

4. 集合 (Set) - `{}` 或 `set()`

- **特点**：无序、可变、**元素唯一**（自动去重）。
- **定义陷阱**：创建空集合必须使用 `set()`，`{}` 默认是空字典。
- **运算**：支持数学集合运算。交集 `&`、并集 `|`、差集 `-`、对称差集 `^`。
- **操作**：`add(x)`（添加）、`update(seq)`（合并）、`remove(x)`（删除，不存在报错）、`discard(x)`（删除，不存在不报错）。

四、 函数

1. 函数基础

- **定义**：使用 `def` 关键字，以冒号 `:` 结尾，依靠**缩进**划分函数体。
- **返回值**：使用 `return`。不写 `return` 时隐式返回 `None`；支持返回多个值（底层自动打包为元组）。

2. 变量作用域

- **局部变量**：函数内部定义，仅在函数内有效。
- **全局变量**：函数外部定义。函数内可读取，若要在函数内修改全局变量，必须使用 `global` 关键字声明。

3. 五种参数传递方式

- **位置参数**
  - **特点**：严格按照函数定义时的参数**顺序**依次传入，形参与实参必须一一对应。
- **关键字参数**
  - **特点**：在调用时通过 `参数名=值` 的形式显式指定（如 `func(name='Alice')`）。
- **默认值参数**
  - **特点**：在定义函数时直接为形参赋予初始值。调用时若不传入该参数，则自动使用默认值。
  - **注意**：定义时**必须**放置在常规位置参数的后面；强烈建议使用**不可变对象**（如数字、字符串、元组）作为默认值，以防多次调用时产生数据污染。
- **可变长位置参数 (*args)**
  - **特点**：用于收集数量未知、多余的位置参数，系统会在函数内部自动将其打包为一个**元组**。
  - **传参方式**：可以直接传入多个用逗号隔开的值（如 `func(1, 2, 3)`）；也可以传入一个已有的列表或元组，并在其变量名前加上 `*` 进行解包操作（如 `func(*my_list)`）。
- **可变长关键字参数 (**kwargs)**

- **特点**：用于收集数量未知、多余的关键字参数，系统会在函数内部自动将其打包为一个**字典。
- **传参方式**：可以连续以 `key=value` 的形式传入（如 `func(a=1, b=2)`）；也可以传入一个已有的字典，并在其变量名前加上 `**` 进行解包操作（如 `func(**my_dict)`）。

4. 高阶概念

- **闭包**：内部函数引用了外部嵌套函数（非全局）的局部变量。
- **递归**：函数调用自身。必须有**退出条件**，Python 默认最大递归深度为 1000 层（可通过 `sys.setrecursionlimit()` 修改）。

五、 类与对象

1. 面向对象基础 (OOP)

- **核心概念**：
  - **面向对象编程 (OOP)**：以“对象”为基本单元，用模型反映现实世界事物特征与规律的抽象思维。
  - **类 (Class)**：创建对象的图纸/代码段。命名遵循 **PEP8 首字母大写 (CapWords)** 规范。
  - **对象/实例 (Object/Instance)**：根据“类”这张图纸造出来的具体实体。
- **核心关键字与方法**：
  - `class`：用于定义类的关键字。
  - `self`：类的实例方法中**必须存在的第一个默认参数**，代表对象自身（类似于 C++/Java 的 `this`）。
  - `__init__(self, ...)`：**构造/初始化方法**。在对象分配内存后自动调用，用于绑定和初始化实例属性。
  - `__del__(self)`：**析构方法**。在对象生命周期结束、被内存释放回收时自动调用。

2. 面向对象三大特性

1. 封装 (Encapsulation)

- **概念**：将属性和方法集中到类中，对外部隐藏内部细节，提升安全性。
- **伪私有变量 (Name Mangling)**：
  - **定义**：在变量或方法名前加**双下划线**（如 `__assets`）。
  - **机制**：Python 底层会自动将其重命名为 `_类名__变量名`（如 `_Company__assets`）。
  - **考点**：Python 中**没有真正的绝对私有**，外部依然可以通过 `实例._类名__变量名` 强制访问。未加下划线的属性默认都是公有的 (public)。

2. 继承 (Inheritance)

- **概念**：子类自动拥有父类的所有属性和方法，极大**降低代码冗余，提高复用性**。
- **单继承**：`class Child(Parent):`
- **多重继承**：`class Child(Base1, Base2):`。支持同时继承多个父类，但易导致代码混乱，需谨慎使用。
- **查找顺序 (MRO)**：在多重继承中，寻找同名方法的底层算法顺序被称为 **MRO (Method Resolution Order)**。
- `super()` **函数**：
  - **作用**：在子类重写方法时，用于安全地**调用父类的同名方法**。
  - **优势**：自动绑定 `self`，自动处理多重继承的 MRO 寻址，无需硬编码父类名称（如 `super().func()`）。

3. 多态 (Polymorphism)

- **概念**：同一种指令（方法调用），作用于不同的对象实例时，表现出不同的行为状态。

- **实现**：通常通过子类**重写 (Override)** 父类的同名方法来实现。就近原则，子类重写后，默认执行子类的方法。

六、 异常

- **异常 vs 错误**：语法错误 (SyntaxError) 在编译期拦截；异常 (Exception) 是程序在**动态运行期间**遇到的特殊危机事件。

1. 异常处理 (`try-except-else-finally`)

- `try:`
  - 存放可能发生异常的“危险代码”。一旦某行报错，该块**剩余代码立刻停止执行**，跳转至 `except`。
- `except Exception as e:`
  - 捕获并处理异常。`e` 是被捕获的**异常对象实例**，包含错误细节。
  - **坑点**：捕获多种异常必须用**元组**打包：`except (ValueError, TypeError):`。极不推荐使用不带类型的“裸 `except:`”。
- `else:`
  - **触发条件**：`try` 块内代码**完全正常执行完毕，未抛出任何异常**时触发。
- `finally:`
  - **触发条件**：**无论是否发生异常，最后必定会执行。**
  - **应用场景**：释放资源、关闭文件句柄、断开数据库连接等硬性清理工作。

2. 自定义异常与主动抛出

- **自定义异常**：必须继承自 `Exception` 基类（或其子类）。通常内部只需写 `pass` 占位即可，核心意义是定制一个符合业务逻辑的“错误标签”（如 `InsufficientFundsError`）。
- `raise` **关键字**：用于在代码中**主动触发/抛出**一个异常对象（如 `raise ValueError("输入不合法")`）。

3. 上下文管理器 (`with` 语句)

- **作用**：简化资源管理代码（如 `with open('file.txt') as f:`）。
- **优势**：即使在文件处理过程中发生异常，也会在离开 `with` 代码块时**自动、安全地关闭文件**，相当于底层替你写好了 `finally: f.close()`。

基本输入输出：

- `print()`：输出指定内容。默认输出后会换行，添加参数 `end=''` 可取消换行。
- `input()`：接收用户键盘输入，默认获取到的数据类型一律为字符串 (`str`)。

七、 文件

1. 文件的打开与关闭

- **基础语法**：`file_obj = open(file_name, mode, buffering)`
- **核心访问模式 (Mode)**：
  - `r`：只读（默认模式）。
  - `w`：写入。若文件存在则**覆盖**，不存在则创建。
  - `a`：追加。内容写在文件**末尾**，不破坏原有数据。
- **缓冲模式 (Buffering)**：
  - `0`：无缓冲（实时写入）。

    - `1`：行缓冲（遇到换行符 `\n` 写入）。
    - `>1` (如 4096)：全缓冲（默认，写满指定字节数后写入）。
- **关闭文件**：`f.close()`。必须执行以释放资源和清空缓冲区。

2. 文件的读与写

- **读操作**：
    - `read(size)`：读取指定字节。不传参则**一次性读取整个文件**，返回大字符串。
    - `readline()`：读取**单行**内容（包含末尾的 `\n`）。
    - `readlines()`：读取**所有行**，返回一个**列表 (List)**，每行是列表的一个元素。
- **写操作**：
    - `write(string)`：将字符串写入文件。**注意**：不会自动添加换行符，需手动加 `\n`。

3. 系统与目录操作 (`os` 模块)

操作前必须 `import os`：

- **文件级**：
    - `os.rename(old, new)`：重命名文件。
    - `os.remove(file)`：删除文件。
- **目录级**：
    - `os.mkdir(dir)`：创建新目录。
    - `os.rmdir(dir)`：删除空目录。
    - `os.getcwd()`：获取当前工作目录 (Get Current Working Directory)。
    - `os.chdir(dir)`：改变当前工作目录 (Change Directory)。

八、 字符串

1. 基础与切片

- **合并与重复**：`'A' 'B'` 自动合并为 `'AB'`；`'Jack' * 3` 变为 `'JackJackJack'`。
- **索引**：从左到右 `0` 开始；从右到左 `-1` 开始（`-1` 代表倒数第一个）。
- **切片** `[start:end:step]`：
    - **口诀**："包左不包右"。例如 `s[1:4]` 截取索引 1, 2, 3，不包含 4。
    - `step`：步长，默认为 1。例如 `s[1:7:2]` 代表隔一个取一个。

2. 格式化输出

- `%` **表达式**：
    - `%s` (字符串), `%d` (十进制整数), `%f` (浮点数)。
    - 考点：保留两位小数使用 `%.2f`。
- `format()` **函数**：
    - `'I am {0}, {1} years old'.format('Jack', 18)`。

3. 常用核心方法

- **清理**：`strip()` 默认去除首尾空格和换行符。
- **统计**：`count(sub)` 统计子串出现的次数。
- **大小写**：`capitalize()` (仅首字母大写)，`lower()` (全小写)，`upper()` (全大写)。
- **查找与判断**：
  - `find(sub)`：找不到返回 `-1`。
  - `index(sub)`：找不到直接 **报错 (ValueError)**。
  - `startswith(sub)` / `endswith(sub)`：判断是否以指定子串开头/结尾。
- **内容类型判断**：
  - `isalnum()`：是否仅由字母**或**数字组成。
  - `isalpha()`：是否纯字母。
  - `isdigit()`：是否纯数字。
- **分割与替换**：
  - `split(sep)`：按分隔符拆分为**列表**。
  - `partition(sep)`：拆分为前、中、后三部分，返回**元组**。
  - `replace(old, new, count)`：默认替换所有出现的旧子串。

九、 正则表达式

1. 核心正则符号

- **匹配字符**：
  - `\d`：数字 (0-9)。
  - `\w`：单词字符（字母、数字、下划线）。
  - `\s`：空白字符。
  - 大写字母 (`\D`, `\W`, `\S`) 代表反义（非数字、非单词、非空白）。
  - `[]`：匹配括号内的任意**一个**字符（如 `[abc]`）。
- **数量匹配 (量词)**：
  - `*`：0次或无限次（可有可无，多多益善）。
  - `+`：1次或无限次（至少有一次）。
  - `?`：0次或1次（最多只能出现一次）。
  - `{m}`：精确出现 m 次。

2. `re` 模块核心方法

- `re.match()` **vs** `re.search()` (必考区别)：
  - `match()`：死板。只从字符串**最开头**匹配，开头不符合直接返回 `None`。
  - `search()`：灵活。在全文中**全局扫描**，找到第一个符合的就返回，全篇没有才返回 `None`。
- **获取结果**：成功匹配后，调用返回对象的 `group()` 方法提取字符串。
- **模式编译**：`re.compile(pattern)` 用于预编译正则图纸，提升重复使用的执行效率。
- **实战扩展**：`re.findall()` 用于找出全文所有匹配项并返回一个**列表**。

十、 Pandas

1. 数据读取与预览

- **读取文件**：`pd.read_csv("文件路径或网络URL")`，专用于读取逗号分隔值文件。
- **数据结构**：`DataFrame` 本质上是由共享索引的多个 `Series` 组合而成的二维数据表。
- **快速预览**：
  - `df.head()`：默认查看前 5 行。
  - `df.tail()`：默认查看末尾 5 行。
  - `df.describe()`：快速输出连续型数值列的统计描述（最大/小值、均值、标准差、分位数）。

2. 数据清洗

- **目的**：处理缺失值、异常值和重复数据，为后续分析或机器学习提供干净的数据基座（模型通常无法直接处理包含 `NaN` 的数据）。
- **核心方法**：
  - `dropna()`：删除包含缺失值的行或列。
  - `fillna()`：使用特定值（如均值、中位数）填充缺失值。
  - `drop_duplicates()`：删除完全重复的数据行。

十一、 数据可视化

1. Matplotlib (`pyplot`)

- **核心概念**：
  - **Figure**：最底层的画布对象（`plt.figure(figsize=(宽,高))` 用于初始化全局尺寸）。
  - **子图**：`plt.subplot()` 可以在同一张大画布中切分多个独立坐标系区域。
- **常用图表**：
  - 折线图：`plt.plot()`（默认将二维数组的各列视为独立的线）。
  - 散点图：`plt.scatter()`。
  - 柱状图：`plt.bar()`。
- **图表定制与输出**：
  - `plt.title()` (标题), `plt.xlabel()` (X轴标签), `plt.legend()` (图例，基于图元的 `label` 参数生成)。
  - 展示与保存：`plt.show()` 弹出图形窗口，`plt.savefig('name.png')` 将图像导出到本地。

十二、 NumPy 基础

1. 数组核心属性

- `ndim`：维度数/秩（如二维数组返回 2）。
- `shape`：形状特征，返回**元组**（如 `(3, 4)` 代表 3行4列）。
- `size`：数组内元素的总个数（形状各维度乘积）。
- `itemsize`：每个元素在内存中占用的**字节数**（例如 `int32` 返回 4字节）。

2. 生成与运算

- **生成**：`np.zeros()` (全零数组), `np.arange(start,` stop, step) (等差序列)。**注意**：多维数组的各行/列长度必须严格一致，否则会退化为一维对象数组。
- **科学计算 (ufunc)**：`np.sin()`, `np.mean()` (算术平均)。底层由 C 语言实现并在连续内存上操作，性能远超原生 Python 列表。
- **矩阵运算**：`np.dot()` 代表线性代数中的矩阵内积乘法；而 `*` 代表逐元素相乘。
- **广播机制 (Broadcasting)**：自动对较小形状的数组进行补齐拉伸，以实现不同形状数组的无缝算术运算。
- **文件存储**：`np.savez()` 将多个数组打包保存为非压缩的 `.npz` 格式；加载后可通过字典键名（如 `r['arr_0']`）提取。

十三、 NumPy 轴操作

1. Axis 轴向解析

- `axis=0`：沿着行方向（纵向扫过）。
- `axis=1`：沿着列方向（横向扫过）。
- `axis=-1`：永远动态指向最内层的最后一个维度。

2. 聚合与状态保留

- **聚合函数**：`sum()`, `mean()`, `max()` 等会导致被操作的维度坍缩消失。
- **保持维度**：加入 `keepdims=True` 参数，坍缩的维度将被保留为 `1`（如变为 `(n, 1)`），便于后续广播计算而不抛出维度错误。
- **累加累积**：`np.cumsum()` (前缀和) 与 `np.cumprod()` (前缀乘积)，操作后维度不坍缩。

3. 带 "arg" 前缀的索引函数

- **核心特性**：不返回极值本身，而是返回**极值所在的索引/坐标**。
- **常用操作**：
  - `np.argmax(arr, axis=1)`：找寻每一行中最大元素所在的列标号。
  - `np.argsort()`：返回排序操作后的索引数组（指导牌），常用于不打散行的间接排序。**警告**：直接对矩阵使用 `np.sort(arr,` axis=0) 会破坏行内数据的绑定关系。

4. 堆叠与拼接

- **在已有轴上拼接 (`concatenate`)**：不增加维度。必须确保非拼接轴的维度严格相等。简写有 `np.hstack()` (水平/列), `np.vstack()` (垂直/行)。
- **创建新轴堆叠 (`stack`)**：结果的维度 `ndim` 一定比原数组多 1 阶。

5. 形状重塑与操作

- **维度调整**：
  - `reshape()`：安全改变维度结构（如利用 `-1` 自动计算长度）。
  - `expand_dims(arr, axis)`：在指定位置插入一个大小为 1 的气泡维度（升维）。
  - `squeeze()`：挤破/消除长度仅为 1 的冗余维度（降维）。
  - `swapaxes(arr, ax1, ax2)`：精准互换两个轴的位置。
  - `transpose()` 或 `.T`：重排矩阵维度视角。
- **展平操作**：`flatten()` 将任意维度的数组碾平为一维（通常采用行优先读取）。

6. 条件逻辑与高级筛选

- `np.where(condition, x, y)`：矢量化的三目运算。若省略 x 和 y，则充当寻址仪，返回符合条件的索引元组。
- **布尔逻辑判定**：
  - `np.any(axis)`：指定轴上存在**至少一个** `True` 即返回 `True`。
  - `np.all(axis)`：指定轴上**全员满足** `True` 才返回 `True`。
- **多条件组合**：在 NumPy 的索引或 `where` 语句中，必须使用按位逻辑运算符 `&` (与)、`|` (或)，并用 `()` 将各个独立条件包裹。
- **边界截断**：`np.clip(arr, min, max)`：将超出范围的值强行修剪/拔高到指定的边界值范围内。
- **唯一值**：`np.unique()` 返回去重并自动升序排列的结果。
- **浮点数比较**：避免使用 `==`，推荐使用 `np.isclose()` 或 `np.allclose()` 以容忍微小的浮点误差

数据分析作业1

1.从键盘输入三个同学的成绩，然后找出最高分。

```
score1, score2, score3 = eval(input("Please Enter score1,score2,score3 : "))
max_score = score1
if score2 &gt; max_score:
    max_score = score2
if score3 &gt; max_score:
    max_score = score3
print(f"The max score: {max_score}")
```

```
The max score: 100
```

2.输入三个同学的成绩，然后大到小排列。

```
score1, score2, score3 = eval(input("Please Enter score1,score2,score3 : "))
if score1 &lt; score2:
    score1, score2 = score2, score1
if score1 &lt; score3:
    score1, score3 = score3, score1
if score2 &lt; score3:
    score2, score3 = score3, score2
print(f"The scores from big to small: {score1}, {score2}, {score3}")
```

```
The scores from big to small: 100, 95, 90
```

3.求出 1000 以内的所有完数，如 6=1+2+3 除了它自身以外的因子之和等于它本身叫完数。

```python
print("1000 以内的所有完数: ", end="")
for i in range(2, 1001):
    total = 0
    for j in range(1, i // 2 + 1):
        if i % j == 0:
            total += j
    if i == total:
        print(i, end=" ")
```

```
1000 以内的所有完数: 6 28 496
```

4.用循环语句求 1+22+333+4444+55555 的和。

```python
total = 0
for i in range(1, 6):
    num = int(str(i) * i)
    total += num
print(f"1 + 22 + 333 + 4444 + 55555 的和: {total}")
```

```
1 + 22 + 333 + 4444 + 55555 的和: 60355
```

5.求出 2000~2100 的所有闰年，条件是能被 4 整除但不能被 100 整除，或者能被 400 整除的是闰年。

```python
for i in range(2000, 2101):
    if i % 100 != 0 and i % 4 == 0:
        print(i, end=" ")
    elif i % 400 == 0:
        print(i, end=" ")
```

```
2000 2004 2008 2012 2016 2020 2024 2028 2032 2036 2040 2044 2048
2052 2056 2060 2064 2068 2072 2076 2080 2084 2088 2092 2096
```

6.输入两个正整数，并求出它们的最大公约数和最小公倍数。

```
a, b = eval(input("Please enter a, b: "))
min = a
max = b
if a &gt; b:
    min, max = b, a
for i in range(min, 1, -1):
    if a % i == 0 and b % i == 0:
        print(i, end=" ")
        break
for j in range(max, min * max + 1):
    if j % a == 0 and j % b == 0:
        print(j, end=" ")
        break
```

```
2 6
```

7.输出 100 以内的所有质数。

```
print("100 以内的所有质数: ", end="")
for i in range(2, 101):
    for j in range(2, i):
        if i % j == 0:
            break
    else:
        print(i, end=" ")
```

```
100 以内的所有质数: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
61 67 71 73 79 83 89 97
```

8.求 100 以内最大的 10 个质数的和。

```
total = 0
count = 0
for i in range(99, 1, -1):
    for j in range(2, i // 2 + 1):
        if i % j == 0:
            break
    else:
```

```
        count += 1
        total += i
        if count == 10:
            break
print(f"100 以内最大的 10 个质数的和: {total}")
```

```
100 以内最大的 10 个质数的和: 732
```

9.求 1~10 的所有偶数的和。

```
total = 0
for i in range(1, 11):
    if i % 2 == 0:
        total += i
print(total)
```

```
30
```

10.将 10~20 不能被 2 或 3 整除的数输出。

```
for i in range(10, 21):
    if i % 3 != 0 and i % 2 != 0:
        print(i, end=" ")
```

```
11 13 17 19
```

```
import re

with open("test.txt", "w") as f:
    while True:
        data = input("Please input your name and socres(English, Math, Physic):")
        if data == "":
            break
        data = data.split(" ")
        if len(data) != 4:
            continue
```

```
            f.write(
                "Name: {0}, English: {1}, Math: {2}, Physic:
{3}\n".format(
                    data[0], data[1], data[2], data[3]
                )
            )
```

```
while True:
    path = input("Please input your path:")
    try:
        file = open(path, "r")
    except FileNotFoundError:
        print("Wrong path")
    else:
        break
data = file.read()
file.close()
info = re.findall(
    r"Name: ([\w-]+), English: ([\d-]{1,3}), Math: ([\d-]{1,3}),
Physic: ([\d-]{1,3})\n",
    data,
)
```

```
import pandas as pd

df = pd.DataFrame(data=info, columns=["Name", "English", "Math",
"Physic"])
df
```

```
df.to_excel("test.xlsx", index=False)
df = pd.read_excel("test.xlsx", na_values="-")
df
```

```
df.columns[df.isnull().any()].tolist()
```

```
df.isnull().sum()
```

```
df.dropna(subset=["Name"], inplace=True, ignore_index=True)
```

```
df.drop_duplicates(subset=["Name"], inplace=True,
ignore_index=True)
```

```
for col in ["Physic", "Math", "English"]:
    df.drop(df[(df[col] &gt; 100) | (df[col] &lt; 0)].index,
inplace=True)
    if df[col].isnull().any():
        df[col] = df[col].fillna(round(df[col].mean(), 1))
```

题目 1：类与对象（面向对象编程）

题目要求：

1. 定义一个父类 `Person`，包含初始化属性 `name` 和 `age`，以及一个方法 `introduce()`。
2. 定义一个子类 `Student`，继承自 `Person`。包含私有属性 `__student_id`（学号）和公有属性 `scores`（存储成绩的字典）。
3. 在 `Student` 类中实现方法 `add_score(subject, score)` 用于添加成绩，并重写（Override）内置方法 `__str__`，使其能格式化输出学生信息和平均分。
4. 实例化对象并演示上述功能。

参考代码及详细注释：

```python
# 1. 定义父类 Person
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"大家好，我叫{self.name}，今年{self.age}岁。")

# 2. 定义子类 Student，继承 Person
class Student(Person):
    def __init__(self, name, age, student_id):
        # 调用父类的初始化方法
        super().__init__(name, age)
        # 定义私有属性（双下划线开头），外部不能直接访问
        self.__student_id = student_id
        # 初始化成绩字典
        self.scores = {}

    # 添加成绩的方法
    def add_score(self, subject, score):
        self.scores[subject] = score
        print(f"已录入 {self.name} 的 {subject} 成绩：{score}分。")

    # 计算平均分的内部方法
    def get_average_score(self):
        if not self.scores:
            return 0
        return sum(self.scores.values()) / len(self.scores)

    # 3. 重写魔术方法 __str__，定义打印对象时的字符串表现形式
    def __str__(self):
        avg = self.get_average_score()
        return f"[学生信息] 姓名:{self.name}, 学号:{self.__student_id}, 平均分:{avg:.1f}"

# 4. 实例化与功能演示
if __name__ == "__main__":
    stu = Student("张三", 20, "S2023001")
    stu.introduce()              # 调用继承自父类的方法
```

```
    stu.add_score("Python", 95)  # 调用自身方法
    stu.add_score("数学", 88)
    print(stu)                   # 触发 __str__ 方法
```

题目 2：组合数据类型综合运用

题目要求：

现有一段文本包含多个以空格分隔的单词。请编写代码：

1. 将字符串（**字符串**）拆分，统计每个单词出现的频次，存储在**字典**中。
2. 使用**集合**去除重复的单词。
3. 将最终的唯一单词与对应频次组合为**元组**的**列表**，并按频次从高到低排序。

参考代码及详细注释：

```python
text = "apple banana orange apple grape banana apple orange kiwi"

# 1. 字符串拆分（切分成列表）
word_list = text.split()

# 2. 使用集合（Set）快速去重，得到唯一的单词集合
unique_words = set(word_list)

# 3. 统计频次并存入字典 (Dictionary)
word_counts = {}
for word in unique_words:
    # 列表的 count() 方法可以统计元素出现次数
    word_counts[word] = word_list.count(word)

# 4. 转换为包含元组 (Tuple) 的列表 (List)
# items() 会返回形如 [('apple', 3), ('banana', 2), ...] 的结构
tuple_list = list(word_counts.items())

# 5. 按照频次（即元组的第1个索引项）从高到低排序
# key=lambda x: x[1] 表示以元组的第二个元素(频次)作为排序依据，reverse=True为降序
sorted_word_list = sorted(tuple_list, key=lambda x: x[1], reverse=True)

print("词频统计及降序排序结果：")
for word, count in sorted_word_list:
    print(f"单词: {word:<10} 频次: {count}")
```

题目 3：文件 + 正则表达式 + 异常 + 字符串 + Pandas 清洗

题目要求：

1. **自定义异常**：定义一个 `InvalidDataError` 异常类。
2. **文件与字符串**：将一段包含脏数据（多余空格、乱码）的文本写入本地文件 `data.txt`，然后再读取出来。
3. **正则与异常**：逐行遍历读取的内容，使用字符串方法清理首尾空格；使用**正则表达式**提取文本中的邮箱地址和成绩。若成绩缺失或为负数，抛出并捕获自定义异常。
4. **Pandas数据清洗**：将提取出的有效字典数据加载为 Pandas DataFrame；进行数据清洗操作：剔除全为空的行（`dropna`），剔除重复数据（`drop_duplicates`），缺失的姓名填充为 "未知"（`fillna`）。

参考代码及详细注释：

```python
import re
import pandas as pd
import numpy as np

# 1. 自定义异常类
class InvalidDataError(Exception):
    def __init__(self, message="捕获到无效数据异常！"):
        super().__init__(message)

# 模拟脏数据
raw_data = """
Name: Alice, Email: alice@test.com, Score: 95
  Name: Bob, Email: bob_123@test.com, Score: 88
Name: , Email: invalid-email, Score: -10
Name: Charlie, Email: charlie@test.com, Score:
Name: Alice, Email: alice@test.com, Score: 95
"""

# 2. 文件写入与读取
file_name = "data.txt"
with open(file_name, "w", encoding="utf-8") as f:
    f.write(raw_data.strip()) # 写入文件

parsed_data = [] # 用于收集有效数据的列表

# 读取文件并逐行处理
with open(file_name, "r", encoding="utf-8") as f:
    for line in f:
        # 字符串清洗：去除首尾多余空白符
        line = line.strip()
        if not line:
            continue

        try:
            # 3. 正则表达式提取邮箱 (匹配 字母/数字/下划线 @ 字母/数字 .字母)
            email_match = re.search(r'[a-zA-Z0-9_]+@[a-zA-Z0-9]+\.[a-zA-Z]+', line)
            email = email_match.group() if email_match else np.nan

            # 正则表达式提取分数 (匹配最后可能存在的数字)
            score_match = re.search(r'Score:\s*(-?\d+)', line)
            if score_match:
                score = int(score_match.group(1))
                if score < 0:
                    # 触发自定义异常
                    raise InvalidDataError(f"成绩不能为负数: {score}")
            else:
                score = np.nan # 缺失分数设为NaN

            # 提取名字（基于字符串截取或正则）
            name_match = re.search(r'Name:\s*([^,]+)', line)
            name = name_match.group(1).strip() if name_match and name_match.group(1).strip() else np.nan

            parsed_data.append({"Name": name, "Email": email, "Score": score})

        except InvalidDataError as e:
```

```
            # 异常捕获机制
            print(f"跳过脏数据行 -> {line} | 原因: {e}")

# 4. Pandas 数据清洗
print("\n--- Pandas 数据清洗 ---")
df = pd.DataFrame(parsed_data)
print("【原始 DataFrame】:\n", df)

# a. 去除所有字段都为 NaN 的完全空行 (how='all') 或 关键字段(Email/Score)为空的行
df_clean = df.dropna(subset=['Email', 'Score'])

# b. 删除重复的行数据
df_clean = df_clean.drop_duplicates()

# c. 缺失值填充：将名字为 NaN 的填充为 "未知"
df_clean['Name'] = df_clean['Name'].fillna("未知")

print("\n【清洗后的 DataFrame】:\n", df_clean)
```

题目 4：Numpy 基础操作函数大全

题目要求:

编写代码，展示Numpy创建数组的所有基础函数（`array`, `arange`, `zeros`, `ones`, `linspace`），打印数组的基础属性（`ndim`, `shape`, `size`, `dtype`），并演示基础通用数学函数（加法、平方根、指数、三角函数）。

参考代码及详细注释:

```
import numpy as np

# 1. 数组的创建方法
print("--- Numpy 数组创建 ---")
# 由列表创建
arr1 = np.array([[1, 2, 3], [4, 5, 6]])
# arange：起点、终点(不包含)、步长
arr2 = np.arange(0, 10, 2)
# zeros：创建一个全0的矩阵，需传入元组代表形状
arr_zeros = np.zeros((2, 3))
# ones：创建一个全1的矩阵
arr_ones = np.ones((2, 2))
# linspace：起点、终点(包含)、元素个数，用于生成线性等分向量
arr_lin = np.linspace(0, 1, 5)

print(f"由 array 创建:\n{arr1}")
print(f"由 arange 创建: {arr2}")
print(f"由 linspace 创建: {arr_lin}")

# 2. 数组的基础属性
print("\n--- Numpy 数组属性 (以arr1为例) ---")
print("秩(维度的数量) arr1.ndim:", arr1.ndim)
print("形状(行数,列数) arr1.shape:", arr1.shape)
print("元素总个数 arr1.size:", arr1.size)
print("数据类型 arr1.dtype:", arr1.dtype)
print("单个元素字节大小 arr1.itemsize:", arr1.itemsize)

# 3. 基础数学通用函数 (ufunc - 矢量化运算)
```

```python
print("\n--- Numpy 基础数学计算 ---")
x = np.array([1, 4, 9])
print("原数组 x:", x)
print("加法 (x + 10):", x + 10)       # 广播机制：所有元素加10
print("平方根 np.sqrt(x):", np.sqrt(x))
print("指数 np.exp(x):", np.exp(x))
print("正弦函数 np.sin(x):", np.sin(x))
```

```
--- Numpy 数组创建 ---
由 array 创建:
[[1 2 3]
 [4 5 6]]
由 arange 创建: [0 2 4 6 8]
由 linspace 创建: [0.   0.25 0.5  0.75 1.  ]

--- Numpy 数组属性 (以arr1为例) ---
秩(维度的数量) arr1.ndim: 2
形状(行数,列数) arr1.shape: (2, 3)
元素总个数 arr1.size: 6
数据类型 arr1.dtype: int64
单个元素字节大小 arr1.itemsize: 8

--- Numpy 基础数学计算 ---
原数组 x: [1 4 9]
加法 (x + 10): [11 14 19]
平方根 np.sqrt(x): [1. 2. 3.]
指数 np.exp(x): [2.71828183e+00 5.45981500e+01 8.10308393e+03]
正弦函数 np.sin(x): [ 0.84147098 -0.7568025   0.41211849]
```

题目 5：Numpy 轴（Axis）操作函数大全

题目要求:

初始化一个三维矩阵（张量）。全面演示 Numpy 针对维度轴（Axis）进行操作的函数:

1. **沿轴聚合**: `sum()`, `mean()`, `max()`
2. **沿轴拼接与堆叠**: `concatenate()`, `stack()`
3. **维度的增减与变换**: `expand_dims()`, `squeeze()`, `swapaxes()`
4. **沿轴寻找最值索引与排序**: `argmax()`, `argsort()`

参考代码及详细注释:

```python
import numpy as np

# 初始化一个 2x2x3 的三维张量 (2块, 2行, 3列)
tensor3d = np.array([
    [[1, 5, 3],
     [8, 2, 9]],

    [[4, 7, 6],
     [0, 3, 5]]
])

print(f"原始三维数组形状: {tensor3d.shape} (块, 行, 列):\n", tensor3d)

```

```python
# 1. 沿轴聚合函数
print("\n--- 1. 沿轴聚合操作 ---")
# axis=0 代表跨越最外层的“块(深度)”进行聚合，将两块中的对应元素相加 (形状变 2x3)
print("axis=0 跨块求和:\n", np.sum(tensor3d, axis=0))
# axis=1 代表跨越每块内部的“行”进行聚合 (形状变 2x3)
print("axis=1 跨行求平均:\n", np.mean(tensor3d, axis=1))
# axis=-1 (等同于axis=2) 代表跨越最内层“列”聚合，keepdims=True 会保留维度不坍缩 (形状变 2x2x1)
print("axis=-1 跨列求最大值并保持维度:\n", np.max(tensor3d, axis=-1, keepdims=True))

# 2. 拼接与堆叠
print("\n--- 2. 沿轴拼接与堆叠 ---")
# 创建另一个同形状的 3D 数组用于拼接
tensor_other = np.ones((2, 2, 3), dtype=int)

# concatenate 沿现有轴拼接，不改变总维度数量 (ndim 仍为 3)
# 沿 axis=1 (行) 拼接，每块内部的行数会从 2 增加到 4，形状变为 (2, 4, 3)
print("axis=1 横向(行)拼接后的形状:", np.concatenate((tensor3d, tensor_other), axis=1).shape)

# stack 沿新创建的轴堆叠，总维度数量(ndim)会加 1，变成 4D
# 沿着最外层 axis=0 堆叠，形状变为 (2, 2, 2, 3)
print("axis=0 新轴堆叠后的形状:", np.stack((tensor3d, tensor_other), axis=0).shape)

# 3. 维度的增减与位置互换
print("\n--- 3. 维度的操作与变换 ---")
# expand_dims: 在指定的 axis=1 位置强行插入一个尺寸为 1 的气泡维度
expanded = np.expand_dims(tensor3d, axis=1)
print(f"在 axis=1 插入新维度，形状从 {tensor3d.shape} 变为: {expanded.shape}")

# squeeze: 将维度尺寸为 1 的多余轴统统挤破/删除
squeezed = np.squeeze(expanded)
print(f"挤压(去除尺寸为1的轴)后，形状恢复为: {squeezed.shape}")

# swapaxes: 精确交换两个轴的位置 (比如将 0轴'块' 和 2轴'列' 互换)
swapped = np.swapaxes(tensor3d, 0, 2)
print(f"交换 0轴和 2轴后，形状从 {tensor3d.shape} 变为: {swapped.shape}")

# 4. 最值索引与排序
print("\n--- 4. 最值索引与排序 (arg系列) ---")
# argmax 返回的是产生最大值的【索引位置】，而不是最大值本身
# axis=2 意味着：在最内层的每一行中寻找，是谁(第几列)最大？
print("沿最内层(列)找最大值的索引 (argmax axis=2):\n", np.argmax(tensor3d, axis=2))
# argsort 返回排序后的原位置索引，这里沿 axis=0 比较两块之间的同位置元素
print("沿块方向升序排列的索引 (argsort axis=0):\n", np.argsort(tensor3d, axis=0))
```

```
原始三维数组形状: (2, 2, 3) (块, 行, 列):
 [[[1 5 3]
  [8 2 9]]

 [[4 7 6]
  [0 3 5]]]

--- 1. 沿轴聚合操作 ---
axis=0 跨块求和:
 [[ 5 12  9]
```

```
 [ 8  5 14]]
axis=1 跨行求平均:
 [[4.5 3.5 6. ]
 [2.  5.  5.5]]
axis=-1 跨列求最大值并保持维度:
 [[[5]
  [9]]

 [[7]
  [5]]]

--- 2. 沿轴拼接与堆叠 ---
axis=1 横向(行)拼接后的形状: (2, 4, 3)
axis=0 新轴堆叠后的形状: (2, 2, 2, 3)

--- 3. 维度的操作与变换 ---
在 axis=1 插入新维度，形状从 (2, 2, 3) 变为: (2, 1, 2, 3)
挤压(去除尺寸为1的轴)后，形状恢复为: (2, 2, 3)
交换 0轴和 2轴后，形状从 (2, 2, 3) 变为: (3, 2, 2)

--- 4. 最值索引与排序 (arg系列) ---
沿最内层(列)找最大值的索引 (argmax axis=2):
 [[1 2]
 [1 2]]
沿块方向升序排列的索引 (argsort axis=0):
 [[[0 0 0]
  [1 0 1]]

 [[1 1 1]
  [0 1 0]]]
```

题目 6：Matplotlib 基础可视化图形大赏

题目要求:

在一个大图板（Figure）中，使用 `subplot` 划分四个子图区域（2x2网格）。

分别绘制：**折线图** (Line)、**散点图** (Scatter)、**柱状图** (Bar) 和 **饼图** (Pie)。

要求解决中文乱码问题，并在每个图中正确配置标题（Title）、X/Y轴标签（Labels）和图例（Legend）。

参考代码及详细注释:

```python
import matplotlib.pyplot as plt
import numpy as np

# 1. 解决中文显示乱码及负号问题
plt.rcParams['font.sans-serif'] = ['SimHei']  # 设置中文字体为黑体
plt.rcParams['axes.unicode_minus'] = False    # 正常显示负号

# 准备测试数据
x = np.arange(1, 6)
y1 = np.array([12, 25, 18, 30, 22])
y2 = np.array([15, 20, 28, 25, 35])
categories = ['A组', 'B组', 'C组', 'D组', 'E组']
pie_data = [20, 30, 25, 25]
pie_labels = ['一季度', '二季度', '三季度', '四季度']
```

```python

# 2. 初始化一个大画布，设置总尺寸
plt.figure(figsize=(10, 8))
plt.suptitle("Python 数据可视化综合大赏", fontsize=16, fontweight='bold')

# --- 子图 1: 折线图 (Line Plot) ---
# subplot(2, 2, 1) 意为: 2行2列网格，占用第1个位置
plt.subplot(2, 2, 1)
# 绘制折线：指定颜色、线型、数据标记点样式、标签(供图例使用)
plt.plot(x, y1, color='blue', linestyle='-', marker='o', label='产品X销量')
plt.plot(x, y2, color='red', linestyle='--', marker='s', label='产品Y销量')
plt.title("折线图 - 销量趋势")
plt.xlabel("月份")
plt.ylabel("销量(万件)")
plt.legend() # 激活显示图例

# --- 子图 2: 散点图 (Scatter Plot) ---
plt.subplot(2, 2, 2)
# s控制点大小，c控制颜色，alpha控制透明度
plt.scatter(x, y1, s=100, c='green', alpha=0.6, label='分布点X')
plt.scatter(x, y2, s=80, c='orange', alpha=0.8, marker='^', label='分布点Y')
plt.title("散点图 - 数据分布")
plt.xlabel("测试次数")
plt.ylabel("得分")
plt.legend()

# --- 子图 3: 柱状图 (Bar Chart) ---
plt.subplot(2, 2, 3)
# 绘制柱状图，使用分类变量作为X轴
plt.bar(categories, y1, color='skyblue', width=0.5)
plt.title("柱状图 - 分组对比")
plt.xlabel("群组")
plt.ylabel("数值")

# --- 子图 4: 饼图 (Pie Chart) ---
plt.subplot(2, 2, 4)
# autopct 用于格式化显示百分比，explode 用于突出显示某一块(这里将第二块略微移出)
explode = (0, 0.1, 0, 0)
plt.pie(pie_data, labels=pie_labels, autopct='%1.1f%%', explode=explode, shadow=True, startangle=90)
plt.title("饼图 - 季度占比")

# 3. 自动调整子图参数，防止标签、标题互相重叠遮挡
plt.tight_layout()

# 4. 将图形推送到屏幕进行展示
plt.show()
```